MASTER PROMPT - KCS INVENTORY MANAGEMENT SYSTEM

VS CODE + FIREBASE-ONLY REVISED EDITION

1. ROLE AND PRIMARY DEVELOPMENT ENVIRONMENT

You are the primary implementation agent for this project, working exclusively in a local Visual Studio Code workspace. Perform all repository inspection, file creation, coding, refactoring, terminal execution, testing, Firebase configuration, documentation, and deployment from VS Code and its integrated terminal. Act as a Senior Full-Stack Software Engineer, Firebase Solution Architect, Firestore Database Architect, UI/UX Designer, Security Engineer, QA Engineer, DevOps Engineer, Technical Writer, and Product Designer. Design, implement, test, document, host, and deploy a complete productionready web application called: KCS Inventory Management System Use Visual Studio Code as the primary development environment for: - Product and implementation planning - Repository creation, inspection, and maintenance - Project scaffolding UI/UX design and visual system implementation - Responsive frontend development - Application and workflow logic - Reusable component development Firebase project integration - Firestore data modeling and typed data access Firebase Authentication - Firestore Security Rules - Firebase Storage Security Rules - Cloud Functions for Firebase - Firebase Cloud Messaging - Firebase App Check - Local development with Firebase Emulator Suite - Unit, integration, end-to-end, permission, and security-rule testing - Browser-based UI verification across desktop, tablet, and mobile layouts - Performance, accessibility, and PWA validation - Firebase Hosting configuration - Preview deployments, production deployment, and rollback documentation - Architecture, setup, administrator, and user documentation Use the VS Code workspace, Explorer, editor, integrated terminal, Source Control panel, Testing interface, debugger, task runner, and a locally launched browser with DevTools to complete the system end to end. Before changing files, inspect the existing repository in VS Code, keep the application runnable, verify work through actual commands and browser testing, and document important architectural decisions. The platform will be used by the ICT Department, warehouse personnel, school management, auditors, and authorized employees of St. Kangoeroe Community School. The system must manage: - ICT assets - Warehouse inventory - Equipment assignments - Equipment loans - Repairs - Maintenance - Inventory movements - Physical inventory audits - Operational reporting - Asset lifecycle history Do not use Manus, Emergent, Lovable, Base44, Replit Agent, Bolt, v0, or another browser-based application builder as the development environment. The complete source code, Firebase configuration, tests, documentation, and deployment scripts must remain in the local Git repository opened in Visual Studio Code.

Do not create a static mockup, basic prototype, disconnected frontend, or demonstration-only interface. Build a scalable, secure, responsive, database-driven enterprise application suitable for daily operational use. Use Firebase exclusively for authentication, operational data, file storage, trusted backend logic, notifications, local emulation, hosting, and production deployment. Do not use Supabase, Appwrite, MongoDB Atlas, external backend-as-a-service products, external frontend hosting providers, external serverless-function platforms, external databases, or provider-specific deployment workflows. The development agent may use maintained frontend libraries from the approved stack, but all persistent application services and trusted backend operations must remain on Firebase. Do not generate configuration files or deployment instructions for Vercel, Netlify, Cloudflare Pages, Render, Railway, AWS Amplify, or any other non-Firebase platform.

2. PROJECT VISION

Create a centralized Enterprise Asset Management and Inventory Management platform for St.

Kangoeroe Community School. The platform must provide complete visibility and control over: - ICT assets - Warehouse inventory - School equipment - Spare parts - Equipment assignments - Equipment borrowing - Equipment returns - Repairs Preventive maintenance - Asset locations - Asset responsibility - Warranty status - Inventory movements - Physical inventory audits - Asset lifecycle history Monthly operational reporting The application must reduce manual paperwork, prevent inventory loss, improve accountability, support faster equipment tracking, and maintain an accurate digital history for every asset.

3. CORE PRODUCT PRINCIPLES

The application must be: - Secure - Reliable - Scalable - Fast - Responsive Bilingual - Professionally organized - Easy for non-technical employees to use - Suitable for long-term institutional use - Designed around traceability and accountability - Capable of expanding without major database restructuring Every important transaction must be traceable. Every asset must have a complete history. Every inventory movement must create an audit record. Operational clarity must always take priority over visual decoration.

4. PRIMARY SYSTEM MODULES

The system must contain the following major modules:

18. Progressive Web App and mobile installation support

5. BRANDING

Use the official St. Kangoeroe Community School branding throughout the application. The official school logo will be uploaded separately. Display the school logo on: - Login page - Registration page - Password recovery page Application sidebar

- Main header
- Loading screen
- Mobile splash screen
- User portal
- PDF reports
- Printed reports
- Exported forms
- Email templates
- System-generated documents Do not permanently embed a temporary

logo. Create a reusable school branding configuration so an administrator can update:

- School logo
- School name
- Address
- Telephone number
- Email address
- Website
- Campus photograph
- Primary brand color
- Secondary brand color
- Report header information

6. COLOR PALETTE

Use colors derived from the official school logo. Initial color palette: - Primary Green: #1FAE5B - Secondary Green: #6BCB77 - Gold: #F4C430 - Light Gold: #FFD95A - Main Background: #F5FAF7 - Primary Text: #1B1B1B - Suc3
cess: #22C55E - Warning: #F59E0B - Danger: #EF4444 - Information: #3B82F6 Use green as the primary operational color and gold as a controlled accent color. Maintain sufficient text contrast and accessibility. The administrator must be able to update the primary branding colors later without rebuilding the application.

7. UI AND VISUAL DESIGN

Create a premium enterprise dashboard influenced by: - Microsoft 365 Admin Center - ServiceNow - Atlassian - Linear - Notion - Stripe Dashboard - IBM enterprise software - Apple interface principles The interface must feel: - Clean Premium - Professional - Minimal - Spacious - Modern - Structured - Trustworthy - Highly organized - Enterprise-grade Avoid: - Cluttered layouts - Excessive gradients - Excessive animations - Weak contrast - Small unreadable text - Decorative elements that interfere with operational work - Excessive transparency
- Consumer-style layouts that reduce data visibility Use consistent: - Spacing
- Typography - Card dimensions - Button hierarchy - Form structure - Table
structure - Status indicators - Icons - Navigation behavior

6A. COMPLETE MULTI-THEME COLOR SYSTEM

This section expands and supersedes the single initial palette in Section 6 for application theming. The original school-branding requirements remain applicable. Implement a complete multi-theme color system for the KCS Inventory Management System.

System default: Theme 4 - KCS Forest Gold. It provides the strongest KCS branding and clearest navigation contrast. Recommended low-distraction option: Theme 3 - KCS Soft Sage for staff who work in the inventory system for long periods.

The application must support five selectable visual themes:

1. Theme 1 - KCS Golden Cream
2. Theme 2 - KCS Glass Campus
3. Theme 3 - KCS Soft Sage
4. Theme 4 - KCS Forest Gold
5. Theme 5 - KCS Azure Intelligence

6A.1 Scope and implementation requirements

The active theme must affect the entire application consistently, including: Login and registration pages; Dashboard; Sidebar navigation; Top navigation bar; Dashboard statistics cards; Forms and input fields; Data tables; Charts and graphs; Buttons; Dropdown menus; Modal windows; Notifications; Status badges; QR-code screens; Inventory records; Asset detail pages; Borrowing and return workflows; Maintenance and repair pages; Reports; User management; Settings; Mobile navigation; Loading states; Empty states; Error pages.

Create reusable semantic design tokens instead of hard-coding colors inside individual components. Use CSS custom properties as the main theming mechanism and connect them to Tailwind CSS or the existing styling system. Do not duplicate components for different themes. All components must use the same semantic variables and update automatically when the theme changes.

Store the selected theme in the authenticated user settings so it remains active after logout, login, refresh, or reopening the application. Also store a local fallback to support the first render and prevent a flash of an incorrect theme.

6A.2 Theme 1 - KCS Golden Cream

Theme ID: kcs-golden-cream
Display name: KCS Golden Cream

Visual direction: A warm, prestigious school identity using cream surfaces, KCS green, gold accents, soft shadows, and subtle glass effects. It must feel welcoming, refined, institutional, and premium.

Palette group	Values
Primary brand colors	Primary Green #1D9431; Dark Green #0E6F2D; Medium Green #3C9D46; Light Green #8EC47D; Pale Green #E8F1D8
Gold colors	Primary Gold #FBC71A; Golden Yellow #F7CF51; Dark Gold #E5AE1E; Pale Gold #FCEDBF
Background and surfaces	Main Background #FCF9EF; Primary Card Surface #FDF7E7; Secondary Surface #FCF4DD; Highlight Surface #FCF2D1; Soft Border #DED6BB; Light Border #ECE8D9
Text	Primary Text #1B1E20; Secondary Text #666961; Muted Text #99968B; Text on Primary #FFFFFF
Status colors	Success #209D54; Warning #F6B900; Repair/Orange #F57C16; Danger #E34F3F; Disabled #AEB1AA

```
Primary button gradient: linear-gradient(180deg, #32A441 0%, #16872C 100%)
Gold accent gradient: linear-gradient(145deg, #F7CF51 0%, #FBC71A 55%, #E5AE1E 100%)
Page background: radial-gradient(circle at 75% 45%, rgba(251, 199, 26, 0.24), transparent 45%),
linear-gradient(135deg, #FDFBF6 0%, #FBF6EA 42%, #FCE8AB 75%, #FCDD75 100%)
Cards: background rgba(253, 251, 246, 0.82); border rgba(229, 195, 116, 0.45); backdrop blur 18px; border radius 18px; subtle warm shadows.
```

6A.3 Theme 2 - KCS Glass Campus

Theme ID: kcs-glass-campus
Display name: KCS Glass Campus

Visual direction: A modern glassmorphism interface displayed over a blurred school-campus photograph. Use cool translucent white surfaces, strong KCS green accents, gold highlights, and dark teal text. The photograph must provide context without reducing text contrast or usability.

Palette group	Values
Primary brand colors	Primary Green #188C35; Bright Green #24A943; Medium Green #4CAD61; Deep Green #0C6F2A; Pale Green #DCEEDF
Gold colors	Primary Gold #FDC218; Warm Gold #F4A900; Pale Gold #FFF1BF
Text	Primary Dark Teal #0B2F3A; Secondary Dark Teal #123D45; Secondary Text #52605D; Muted Text #7C8581; Text on Primary #FFFFFF
Glass surfaces	Main Glass rgba(255,255,255,0.72); Strong Glass rgba(255,255,255,0.84); Sidebar Glass rgba(244,247,244,0.76); Muted Glass rgba(230,232,228,0.74); Input Glass rgba(255,255,255,0.68); Glass Border rgba(255,255,255,0.72); Subtle Border rgba(185,195,190,0.48)
Status colors	Success #188C35 / soft #DCF3E2; Warning #F4A900 / soft #FFF1BF; Danger #F04B3F / soft #FFE3DE; Information #3F8DBD; Neutral #727B78

```
Glass card: background rgba(255,255,255,0.72); 1px solid rgba(255,255,255,0.72); blur 18px; saturation 125%; radius 16px; shadow 0 8px 28px rgba(15,35,28,0.12).
Login panel: background rgba(245,247,243,0.70); 2px solid rgba(255,255,255,0.82); blur 22px; radius 42px; shadow 0 24px 70px rgba(11,31,24,0.20).
Primary button gradient: linear-gradient(180deg, #29B94A 0%, #168C35 100%). Gold icon gradient: linear-gradient(145deg, #FFD326 0%, #F4A000 100%).
Background image: use approved campus image; cover; center; optional dark/light overlay; blur background layer only; preserve WCAG-readable contrast.
```

6A.4 Theme 3 - KCS Soft Sage

Theme ID: kcs-soft-sage
Display name: KCS Soft Sage

Visual direction: A calm operational interface using pale sage backgrounds, soft ivory cards, medium green actions, and restrained gold accents. This is the recommended low-distraction theme for prolonged daily inventory work.

Palette group	Values
Green colors	Deep Forest Green #0F2E22; Secondary Dark Green #38544C; Primary Green #268C36; Bright Green #48A851; Light Green #6CBD63; Soft Sage #BDD3BE; Pale Sage #D8E4D6
Gold colors	Primary Gold #F2B822; Light Gold #F9D05A; Pale Gold #FFF1C7
Backgrounds and surfaces	Main Background #F3F5EC; Secondary Background #ECF1E8; Pale Green Surface #E3ECE1; Card Surface #F9FBF6; Warm Cream Surface #EEF0DD; Sage Border #D8E4D6; Strong Border #BDD3BE
Text	Primary Text #0F2E22; Secondary Text #38544C; Muted Text #627C7B; Disabled Text #97A99E; Text on Primary #FFFFFF
Status colors	Success #268C36; Success Bright #48A851; Success Soft #E3F2E4; Warning #F2B822 / soft #FFF1C7; Danger #D32C1D; Secondary Danger #E9815A; Danger Soft #FDE5DF; Information #258CE2 / soft #E2F0FD

```
Card: background rgba(249,251,246,0.90); border rgba(189,211,190,0.70); blur 14px; radius 16px; shadow 0 8px 24px rgba(15,46,34,0.08).
Sidebar gradient: linear-gradient(180deg, rgba(243,245,236,0.96) 0%, rgba(227,236,225,0.92) 100%).
Primary green gradient: linear-gradient(145deg, #6CBD63 0%, #48A851 45%, #268C36 100%).
Gold gradient: linear-gradient(145deg, #F9D05A 0%, #F2B822 60%, #F39A18 100%).
```

6A.5 Theme 4 - KCS Forest Gold

Theme ID: kcs-forest-gold
Display name: KCS Forest Gold

Visual direction: A high-contrast institutional theme with a deep forest-green sidebar, ivory dashboard surfaces, gold active navigation, and strong green operational indicators. This is the system default and must feel formal, stable, authoritative, and closely connected to the school identity.

Palette group	Values
Green colors	Deepest Forest Green #004C25; Sidebar Green #015B2B; Primary Green #0B9247; Medium Green #1D6F3F; Soft Green #459560; Muted Sage #93AD99; Pale Sage #E0E7DC
Gold colors	Primary Gold #F6BE23; Light Gold #F6CE5D; Pale Gold #FFF1C4
Backgrounds and surfaces	Main Background #FCFCF5; Card Surface #F8F8EF; Cream Surface #F5F4E3; Pale Sage Surface #E0E7DC; Default Border #CBD2C9; Muted Border #B4BFB4
Text	Primary Text #102E20; Dark Green Text #163E2A; Secondary Text #4E5E54; Muted Text #78857D; Text on Dark Surface #FFFFFF
Status colors	Success #0B9247 / soft #DDEEDF; Warning #F6BE23 / soft #FFF1C4; Orange #F28C18; Danger #EB654B / soft #F7C9AA; Information #2388D8; Neutral #5E6662

```
Sidebar gradient: linear-gradient(180deg, #01642F 0%, #015B2B 52%, #004C25 100%).
Active navigation gradient: linear-gradient(180deg, #FFD23F 0%, #F6BE23 100%); text #163E2A; border
rgba(255,224,100,0.85); shadow 0 4px 14px rgba(246,190,35,0.28).
Cards: background rgba(252,252,245,0.94); border rgba(203,210,201,0.78); radius 16px; shadow 0 8px 24px
rgba(1,91,43,0.07).
Use white icons and text in the dark sidebar. Use gold for active route, notification counters, selected
filters, and important attention states.
```

6A.6 Theme 5 - KCS Azure Intelligence

Theme ID: kcs-azure-intelligence
Display name: KCS Azure Intelligence

Visual direction: A modern technology-oriented theme inspired by clean AI and digital-platform interfaces. Use blue gradients, white surfaces, soft lavender accents, and dark indigo typography. Keep the KCS logo and school identity. Green is reserved mainly for success states; blue is the primary interface color.

Palette group	Values
Blue colors	Deepest Blue #1355CC; Deep Brand Blue #1366DC; Primary Blue #147AE7; Bright Blue #168DF0; Sky Blue #349BF4; Light Blue #51A2F7; Soft Blue #82ADFB; Lavender Blue #9AB3FD; Pale Blue #B3D5F8
Indigo colors	Deep Indigo #34344F; Primary Indigo Text #3D3D59
Neutrals	Primary White #FFFFFF; Cool Off-White #EDF9FE; Page Background #E3E5ED; Muted Gray #C2C3C8; Secondary Text #707387; Light Border #E3E5ED
Supporting status colors	Success #209D54 / soft #DCF3E2; Warning #F6B900 / soft #FFF1BF; Danger #E34F3F / soft #FFE3DE; Information #147AE7

```
Main application gradient: linear-gradient(180deg, #9AB3FD 0%, #51A2F7 45%, #168DF0 68%, #1366DC 100%).
Primary button gradient: linear-gradient(135deg, #51A2F7 0%, #168DF0 45%, #1366DC 100%).
Accent gradient: linear-gradient(180deg, #9AB3FD 0%, #349BF4 48%, #147AE7 72%, #1355CC 100%).
Cards: background rgba(255,255,255,0.90); 1px solid rgba(179,213,248,0.75); radius 16px; shadow 0 10px 30px rgba(19,102,220,0.10).
Sidebar: white with blue active states or blue gradient. Active navigation may use #147AE7. Text on blue #FFFFFF; text on white #3D3D59.
```

6A.7 Required semantic design tokens

Create the following semantic variables for every theme:

```
--color-background
--color-background-secondary
--color-surface
--color-surface-secondary
--color-surface-elevated
--color-surface-hover

--color-primary
--color-primary-hover
--color-primary-active
--color-primary-soft
--color-on-primary

--color-secondary
--color-secondary-hover
--color-secondary-soft
--color-on-secondary

--color-accent
--color-accent-hover
--color-accent-soft

--color-text-primary
--color-text-secondary
--color-text-muted
--color-text-disabled
--color-text-inverse

--color-border
--color-border-subtle
--color-border-strong
--color-focus-ring

--color-success
--color-success-soft
--color-warning
--color-warning-soft
--color-danger
--color-danger-soft
--color-info
--color-info-soft

--color-sidebar-background
--color-sidebar-text
--color-sidebar-muted
--color-sidebar-active-background
--color-sidebar-active-text
--color-sidebar-hover

--color-input-background
--color-input-border
--color-input-text
--color-input-placeholder

--color-chart-1
--color-chart-2
--color-chart-3
--color-chart-4
--color-chart-5
--color-chart-6
--color-chart-7
--color-chart-8

--shadow-card
--shadow-dropdown
--shadow-modal
--radius-card
--radius-button
--radius-input
```

6A.8 Settings > Appearance theme selector

Add a Theme section to Settings > Appearance. Display each theme as a visual preview card containing: theme number; theme name; miniature sidebar preview; background preview; primary, secondary, and accent colors; sample button; sample card; and sample success, warning, and danger badges.

Theme options: 1. KCS Golden Cream; 2. KCS Glass Campus; 3. KCS Soft Sage; 4. KCS Forest Gold; 5. KCS Azure Intelligence.

1. Apply the selected theme immediately without refreshing the page.

2. Show a checkmark on the active theme.

3. Save the selection locally.

4. Save it to the authenticated user profile in Firebase.

5. Synchronize it across the user devices.

6. Fall back to Theme 4 when no preference exists.

7. Prevent a flash of the default or incorrect theme during application loading.

Theme preference is a user-interface preference, not an authorization control. Firestore Security Rules must permit a user to update only allowed appearance-preference fields in that user's own profile unless an administrator is performing an authorized support action.

6A.9 Accessibility, reporting, and quality rules

- Meet WCAG AA contrast requirements for normal text.

- Do not use color as the only status indicator.

- Include text labels and icons for success, warning, danger, and information.

- Maintain clear keyboard focus indicators.

- Support hover, active, disabled, selected, loading, and error states.

- Preserve readability in dense tables.

- Maintain sufficient contrast in charts and legends.

- Verify desktop, tablet, and mobile layouts.

- Respect prefers-reduced-motion.

- Avoid excessive transparency behind small text.

- Do not apply decorative gradients to body text.

- Ensure print-friendly reports use a neutral white background.

- Ensure generated PDF reports remain readable regardless of active theme.

- For Theme 2, reduce transparency or add a stronger surface overlay whenever contrast would otherwise fail.

- All theme names and appearance controls must be available in Dutch and English translation resources.

6A.10 Acceptance criteria

- Theme 4 is active for first-time users and when no valid saved preference exists.

- Theme 3 is visibly labeled as the recommended low-distraction option.

- All five themes work on login, dashboard, tables, forms, dialogs, charts, mobile navigation, loading, empty, error, and permission-denied states.

- No component contains theme-specific hard-coded colors outside the centralized theme definitions.

- Theme changes are immediate and persist locally and in Firebase.

- No incorrect-theme flash occurs during authentication and application bootstrap.

- Charts re-render with the active palette and remain distinguishable without relying only on color.

- Printed and PDF reports remain neutral, professional, and readable.

- Automated tests cover selection, persistence, fallback, and first-render behavior.

8. GLASSMORPHISM DESIGN SYSTEM

Use a controlled premium glassmorphism style. The official school campus photograph may be uploaded and used as an optional background. When the campus photograph is enabled: - Apply a soft blur - Apply a subtle dark or light overlay - Maintain readable text contrast - Keep tables and forms clearly visible - Allow the administrator to disable the photographic background Recommended glass card styling: .glass-card { background: rgba(255, 255, 255, 0.18); backdrop-filter: blur(20px); -webkit-backdrop-filter: blur(20px); border: 1px solid rgba(255, 255, 255, 0.28); box-shadow: 0 12px 40px rgba(0, 0, 0, 0.12); border-radius: 24px; } Glass cards should include: - Frosted glass surfaces - Soft shadows - Rounded corners - Clear borders - Smooth hover states - Consistent spacing - Strong content hierarchy For large data tables, reports, and long forms, prioritize readability over glass effects.

9. RESPONSIVE DESIGN

The entire system must work on: - Desktop computers - Laptops - Tablets Mobile phones Desktop must be the primary operational layout. Tablet and mobile views must support: - QR-code scanning - Barcode scanning - Asset verification - Inventory audits - Quick asset lookup - Borrow return processing - Adding photographs Recording service information - Updating asset locations - Capturing signatures
- Completing maintenance checklists Use responsive: - Tables - Cards - Filters -
Dialogs - Drawers - Navigation - Forms - Charts - Action buttons Large desktop tables should become structured cards or compact lists on mobile devices.

10. AUTHENTICATION

Create a secure authentication system with: - Login - Registration - Forgot password - Password reset - Remember me - Email verification - Logout - Secure session handling - Session timeout - Account activation - Account deactivation Profile management - User settings - Password change - Last login information
- Login attempt tracking
Administrators must be able to decide whether public registration is enabled. When public registration is disabled, new accounts must be created or approved by an administrator. Do not automatically grant operational access to newly registered users. A new account must receive an approved role before accessing protected modules.

11. USER ROLES

Create role-based access control for the following initial roles. Administrator Full system access, including: - Users - Roles - Permissions - Settings - Reports - Audit logs - Backups -
All operational modules ICT Manager Manage: - ICT assets - ICT staff - Repairs
Maintenance - Asset assignments - ICT reports - Inventory audits Warehouse Manager Manage: -
Warehouse inventory - Incoming stock

- Outgoing stock
- Transfers
- Stock corrections
- Minimum stock levels
- Warehouse reports
- Inventory audits

ICT Staff Operational access to: - Assets - Repairs - Maintenance - QR-code scanning -
Asset updates - Assigned technical tasks Warehouse Staff Operational access to: - Stock registration - Stock movements Equipment issuing - Equipment returns - Warehouse counts - QR-code scanning
- Barcode scanning

School Management Read and reporting access to: - Dashboard - Management reports - Asset summaries - Inventory statistics - Audit results - Read-Only Auditor

Read-only access to: - Inventory records - Asset history Audit logs - Reports

- Physical audit results Administrators must be able to create custom roles.

Permissions must support:

- View
- Create
- Edit
- Delete
- Approve
- Export
- Print
- Assign
- Archive
- Restore
- Manage settings
-
- Perform audits
- Correct discrepancies

12. APPLICATION NAVIGATION

Create a collapsible sidebar containing: - Dashboard - ICT Assets - Inventory Categories - Locations - Departments - Warehouse - Borrow Management - Service Forms - Repairs - Maintenance - Inventory Movements

- Inventory Audits - Reports - Notifications - Users - Roles and Permissions Activity Logs - Settings - Logout

The sidebar must: - Display the school logo Highlight the active page - Support collapsed and expanded modes

- Show only role-appropriate menu items
- Work properly on mobile devices
- Support Dutch and English labels
- Display notification badges where relevant

13. DASHBOARD

Create a role-aware dashboard. Display summary cards for: - Total registered assets - Available assets - Assigned assets - Borrowed assets - Assets under repair - Assets under maintenance - Damaged assets - Lost assets - Disposed assets - Archived assets - Items below minimum stock - Warranties expiring soon inventory audits - New registrations - Today’s activities

- Monthly inventory movements Include: - Recent inventory movements - borrow requests - Upcoming return dates - Upcoming maintenance - Warranty expiration overview - Latest audit results - Notifications - Quick actions - Charts

- Graphs Quick actions must include: - Add asset - Add inventory item

- Scan QR code
- Scan barcode
- Create borrow request
- Register return
- Start inventory audit
- Record stock movement
- Generate report Charts may include:
- Assets by category
- Assets by status
- Assets by location
- Inventory movements over time

- Repairs by month
- Borrowed versus returned equipment
- Warranty expiration timeline
- Audit discrepancy trends
- Low-stock categories Dashboard information must adjust according to the

current user's role and permissions.

14. ASSET AND INVENTORY CATEGORIES

Support the following initial categories: - Desktop computers - Laptops - Monitors - Keyboards - Mice - Printers - Routers - Network switches - Access points

- UPS devices - SSDs - HDDs - RAM - Projectors - Smartboards - Digital board

modules - Controllers - Webcams - Headsets - Speakers - Batteries - Cables Tablets - Phones - Office furniture - Server racks

- Tools
- Spare parts
- Servers
- Accessories
- Other school equipment Administrators must be able to:
- Add categories
- Edit categories
- Archive categories
- Restore categories
- Create subcategories
- Add category-specific custom fields
- Define whether an item is serialized or quantity-based
- Define minimum stock levels
- Assign maintenance requirements
- Assign default lifecycle assessment rules Do

not hard-code the platform so that it only works with the initial categories.

15. ASSET TYPES

Support two inventory models. Serialized Assets Used for individually traceable equipment such as: - Laptops

- Desktop computers - Printers - Projectors - Smartboards - Servers - Network

devices - Tablets - Phones Each serialized asset must have its own: - Asset record - QR code - Barcode - Serial number - Status - Location - Assignment Condition

- History

Quantity-Based Inventory Used for consumables, components, and bulk stock such as: - Cables - Batteries - Connectors - Toner - Small accessories - Spare parts - Office supplies Quantity-based items must support: - Quantity on hand

- Quantity reserved - Quantity available - Minimum stock level - Reorder level Reorder quantity - Unit of measure - Storage location - Stock movement history

16. ASSET RECORD

Every serialized asset must have a complete digital asset card. Include: - Internal asset ID - Asset number - QR code - Barcode - Serial number - Asset name - Description - Brand - Model - Category - Subcategory - Location - Room or classroom - Building - Department - Assigned user - Responsible employee Current status

- Condition
- Purchase date

- Warranty start date
- Warranty expiration date
- Supplier
- Manufacturer
- Photos
- Attachments
- Notes
- Created by
- Created date
- Last modified by
- Last modified date Optional technical fields:
- MAC address
- IP address
- Hostname
- Operating system
- Processor
- RAM
- Storage capacity
- Device specifications
- Network information
- License information
- Technical notes The asset card must also display:
- Assignment history
- Location history
- Borrow history
- Service history
- Repair history
- Maintenance history
- Inventory movement history
- Audit history
- Status history
- Attached documents
- Related accessories

17. ASSET STATUS

Use the following standardized statuses: - Available - Assigned - Borrowed Under Repair - Under Maintenance - Reserved

- Lost
- Missing
- Damaged
- Disposed
- Archived The administrator must be able to configure additional statuses.

Status changes must record:

- Previous status
- New status
- User who made the change
- Date and time
- Optional reason
- Related transaction
- Activity log entry

18. ASSET CONDITION

Track asset condition separately from operational status. Condition options: New - Excellent - Good - Fair - Poor - Defective - Beyond Repair Condition changes must be stored in the asset history. Users must not treat condition and operational status as the same data field.

19. LOCATIONS AND DEPARTMENTS

Create a hierarchical location structure. Support: - Campus - Building

- Floor
- Department
- Office
- Classroom
- Laboratory
- Warehouse
- Storage room
- Shelf
- Rack
- Specific storage position Administrators must be able to create, edit,

archive, and restore locations. Every asset and inventory item must have a current location. Create configurable department records such as:

- ICT
- Administration
- Finance
- Mathematics
- Languages
- Science
- Management
- Warehouse
- Other school departments

20. EMPLOYEE AND USER ASSIGNMENT

Assets may be assigned to: - Teachers - Administrative employees - ICT personnel - Warehouse personnel - Departments - Classrooms - Offices - Other authorized users Assignment records must include: - Asset - Assigned person Department - Location - Assignment date - Expected end date when applicable
- Assigned by - Condition at assignment - Notes

- Digital or physical signature
- Return date
- Condition at return
- Returned to
- Return notes The system must maintain complete assignment history. Assignments must never overwrite historical assignment information.

21. QR-CODE AND BARCODE MANAGEMENT

Generate a unique QR code for every serialized asset. Support: - QR-code generation - Barcode generation - QR-code printing - Barcode printing - Custom label templates - Bulk label printing - Camera scanning - Mobile scanning - USB barcode scanners - Quick asset lookup after scanning Scanning a code must open the relevant asset or inventory record. Authorized users must be able to perform quick actions after scanning: - View asset - Update status - Change location
Mark audit verification

22. BORROW MANAGEMENT

Create a complete equipment borrowing module. Support borrowing for: - Laptops - Projectors - Smartboards - Digital board modules - Printers - Tablets - Accessories - Other equipment Each borrow record
must include: - Borrow reference number - Asset or equipment - Borrower Borrower type - Department - Reason - Borrow date - Expected return date

- Actual return date - Approved by - Issued by - Received back by - Status
- Condition before issue - Condition after return - Borrower signature - Staff
signature - Notes - Attachments Borrow statuses: - Draft - Pending Approval Approved - Rejected - Issued - Partially Returned - Returned - Overdue - Cancelled The system must automatically identify overdue equipment. Generate: Borrow agreement - Laptop loan form - Equipment issue form - Return receipt
- Overdue equipment report Forms must be printable and exportable as PDF.

24. SPECIALIZED EQUIPMENT FORMS

Create configurable forms for the following equipment types. Laptop Registration Form Include: - Asset number - Serial number - Brand Model - Specifications - Status - Condition - Warranty - Assigned user - Department - Location - Accessories - Charger information - Notes Printer Form Include: - Asset number - Serial number - Brand - Model - Printer type - Network information - Toner type - Toner status

- Location
- Assigned department
- Maintenance schedule
- Fault history
- Page counter when available
- Notes

Smartboard and Digital Module Form Include: - Asset number - Serial number Brand - Model - Classroom
- Related modules - Controllers - Cables - Accessories
- Firmware information - Installation date - Maintenance history - Technical
status - Notes All specialized forms must connect directly to the primary asset record. Do not create disconnected or duplicate asset records.

25. REPAIR MANAGEMENT

Create a separate but connected repair module. Each repair record must include:
- Repair number - Related asset - Problem description - Diagnosis - Repair
priority - Assigned technician - External repair company when applicable - Date received - Repair start date - Expected completion date - Actual completion date
- Repair notes - Parts replaced - Repair outcome

- Attachments
- Before and after photos
- Warranty repair indicator
- Status Repair statuses:
- Reported
- Diagnosing
- Awaiting Approval
- Waiting for Parts
- In Repair
- Testing
- Completed
- Returned to User
- Unrepairable
- Cancelled When an asset enters repair, its asset status must automatically

change to Under Repair. When a repair is completed, the user must select the asset’s new status and condition.

26. PREVENTIVE MAINTENANCE

Create a maintenance module for planned and recurring work. Support: - Annual inspections - Preventive maintenance - Device cleaning - Firmware updates Antivirus checks - Windows updates - Printer

maintenance - UPS battery checks
- UPS battery replacement - Network maintenance - Smartboard calibration Warranty checks -
Equipment testing Each maintenance schedule must include:
- Related asset or category - Maintenance type - Frequency - Last maintenance
date - Next maintenance date - Assigned employee

- Instructions
- Checklist
- Completion status
- Completion notes
- Attachments Support recurrence such as:
- Weekly
- Monthly
- Quarterly
- Every six months
- Annually
- Custom interval Automatically generate reminders for upcoming and overdue maintenance.

27. INVENTORY AND WAREHOUSE MANAGEMENT

Create inventory management for both serialized and quantity-based stock. Support: - Stock registration - Stock receiving - Stock issuing - Transfers - Returns
- Reservations - Corrections - Damaged stock - Lost stock - Disposal - Minimum
stock control - Reorder warnings - Storage location tracking Each quantity-based inventory item must include: - Item ID - Item code - Barcode - Item name - Description - Category - Unit of measure - Quantity on hand - Quantity reserved
- Quantity available - Minimum quantity - Reorder quantity

- Warehouse
- Storage location
- Supplier
- Photo
- Notes The system must prevent inventory from becoming negative.

28. INVENTORY MOVEMENTS

Track every inventory movement. Movement types: - Incoming - Outgoing
- Transfer - Assignment - Borrow - Return - Repair Transfer - Maintenance
Transfer - Stock Correction - Damage - Loss - Disposal - Audit Adjustment Every movement must include: - Movement reference number - Movement type - Item or asset - Quantity when applicable - Source location - Destination location Related employee - Related department - Reason - Date and time - Performed by
- Approved by when required - Notes - Attachments Movement records must not
be silently deleted. Corrections must create compensating records and preserve the original transaction history.

29. RANDOM INVENTORY AUDIT MODULE

The inventory audit module is a critical system component. The system must generate random asset verification lists for daily, weekly, or monthly inventory checks. The administrator must be able to configure: - Audit name - Audit frequency - Number of items - Percentage of total items - Asset categories Inventory categories - Locations - Departments - Statuses - Serialized assets only - Quantity-based items only - Mixed selection - Excluded records - Random
selection rules - Assigned audit employees - Audit deadline Examples: - Select

10 random laptops - Select 5 printers - Select 3 monitors - Select 12 office chairs - Select 50 mixed ICT assets - Select 5% of all assets in one building - Select all assets in one randomly selected classroom The selection process must: - Use an unbiased random selection method - Avoid unnecessary repeated selection of recently audited assets when configured - Store the generated list permanently - Prevent the list from changing after the audit begins - Record who generated the audit - Record the generation date and time For every selected asset, the auditor must record one of the following results: - Verified - Missing - Damaged - Wrong Location - Wrong Assigned User - Needs Repair

- Needs Maintenance
- Data Incorrect
- Not Accessible The auditor must also be able to:
- Scan the asset QR code
- Confirm the current location
- Confirm the responsible user
- Update the condition
- Add notes
- Add photographs
- Report discrepancies After completion, generate an audit report containing:
- Total selected assets
- Total verified
- Total missing
- Total damaged
- Total in the wrong location
- Total requiring repair
- Total with incorrect records
- Completion percentage
- Discrepancy rate
- Auditor information
- Audit date
- Corrective actions Audit adjustments must require appropriate permissions and must create audit logs.

30. GLOBAL SEARCH

Create a fast global search feature. Allow searching by: - Asset ID - Asset number - Item code - Barcode - QR code - Serial number - Asset name - Brand - Model - Category - Subcategory - Location - Classroom - Building

- Assigned employee
- Responsible employee
- Department
- Status
- Condition
- Supplier
- Borrow reference
- Repair number
- Audit number Support:
- Partial matches
- Exact matches
- Filters
- Sorting
- Saved filters
- Recent searches
- Search suggestions

31. FILTERING AND DATA TABLES

All major data pages must include: - Search - Filters - Sorting - Pagination Column selection - Saved views - Bulk actions - Export - Print - Responsive table display Useful filters include: - Category - Status - Condition - Location
- Department - Assigned user - Warranty status - Purchase date - Maintenance
status - Repair status - Borrow status - Audit status Users must only see filters and actions that their role is permitted to use.

32. REPORTING

Create a comprehensive reporting module. Support: - Daily reports - Weekly reports - Monthly reports - Custom date-range reports Report types must include: - Complete inventory list - ICT assets by category - ICT assets by location
- ICT assets by classroom - ICT assets by department - ICT assets by assigned
employee - Assets by status - Assets by condition - Available assets - Assigned assets - Borrowed equipment - Overdue equipment - Returned equipment - Items added - Items issued - Items received - Inventory movements - Low-stock items
- Damaged assets - Lost assets - Disposed assets - Repairs - Maintenance - Warranty
expiration - Supplier activity - Inventory audit results
report The monthly inventory report must summarize:
- Total ICT inventory - Total warehouse inventory - New items added - Items
issued

- Items borrowed
- Items returned
- Items under repair
- Items under maintenance
- Damaged items
- Defective items
- Lost items
- Disposed items
- Inventory by category
- Inventory by location
- Inventory by department
- Warranty status
- Low-stock warnings
- Audit activity
- Reports must support:
- PDF export
- Excel export
- CSV export
- Print-friendly format
- School logo
- School information
- Report title
- Date range
- Generated by
- Generated date and time
- Page numbering Allow authorized users to schedule monthly report generation.

33. NOTIFICATIONS AND REMINDERS

Create an in-application notification center. Notify authorized users about: Warranty expiration - Upcoming maintenance - Overdue maintenance - Borrow return dates - Overdue borrowed equipment - Low stock - Repair completion

Audit deadlines - Audit discrepancies - Missing assets

- System activity requiring attention Support:
- Read and unread states
- Notification categories
- Notification preferences
- Role-based notifications
- Email notifications when configured
- Push notifications when configured
- Notification history

35. AI INVENTORY ASSISTANT

Include an AI assistant that can answer authorized inventory questions using current system data. Example questions: - Which laptops are currently available? - Which devices are under repair? - Which assets need maintenance this month? - Which warranties expire this month? - Which monitors are located in classroom 5? - Which assets do not have a responsible employee? - Show all HP laptops. - Which assets were missing during the last audit? - Which borrowed devices are overdue? - Generate a monthly inventory summary. - Show all assets assigned to a specific teacher. - Show all assets assigned to a department.
- Recommend assets for replacement based on age, warranty status, condition, and repair history. The AI assistant must: - Respect role-based permissions Never expose unauthorized records - Use actual database information - Clearly indicate when information is unavailable - Provide links to relevant records Support Dutch and English - Generate summaries without silently modifying records - Require explicit confirmation before initiating any system action The AI assistant must never bypass application permissions.

36. ACTIVITY LOGS AND AUDIT TRAIL

Create immutable audit logs for important system activity. Record: - User Action - Module - Record type - Record ID - Previous values - New values

- Date and time
- Device or session information when available
- Reason when required Audit events must include:
- Login
- Failed login
- Asset creation
- Asset update
- Asset assignment
- Status change
- Location change
- Borrow approval
- Equipment issue
- Equipment return
- Repair update
- Maintenance completion
- Inventory movement
- Audit result
- User creation
- Role change
- Permission change
-
- Data export
- Record archival Audit logs must be searchable and exportable. Only authorized users may view audit logs.

37. DATA ARCHIVAL AND DELETION

Do not permanently delete operational records by default. Use lifecycle states such as: - Active - Inactive - Archived - Disposed Provide controlled archive and restore functions. Permanent deletion must: - Be limited to administrators

- Require explicit confirmation - Be blocked when related transactions exist

- Be recorded in the audit log

38. SETTINGS

Create a complete settings area. General Settings - School name - School address - Contact information - Logo

- Campus photograph - Default language - Date format - Time format - Time

The language change must update: - Navigation - Buttons - Forms Validation messages - Notifications - Reports - Status labels - System messages

- Email templates - Printable forms Do not translate only the main navigation.

Appearance Settings - Light theme - Dark theme - System theme

- Campus background enabled or disabled
- Glass effect intensity
- Primary color
- Accent color

Inventory Settings - Asset number format - Barcode format - QR-code settings Default statuses - Default conditions - Minimum stock rules - Category settings

- Required fields

Audit Settings - Default audit frequency - Default sample size - Default sample percentage - Random selection rules - Recently audited exclusion period Approval requirements Notification Settings - Notification types - Email notifications - Push notifications - Reminder periods - Escalation rules permissions

Security Settings - Session timeout - Password requirements - Registration policy

- Email verification - Account lockout - Login attempt limits

Backup Settings - Backup status - Backup schedule - Last backup - Data export

- Recovery information

39. SECURITY REQUIREMENTS

Implement: - Firebase Authentication - Role-based access control - Firestore Security Rules - Secure Firebase Storage Rules - Server-side authorization checks

- Protected routes - Input validation - Output sanitization - Secure API endpoints - Session timeout - Account deactivation - Audit logging - Least-privilege

access - Secure environment variables - Rate limiting where appropriate - File upload restrictions - File size validation - File type validation Do not rely only on hiding interface buttons. Every protected action must also be enforced in the backend and database security rules.

40. BACKUP AND DATA PROTECTION

Prepare the architecture for: - Automated database backups - File backup strategy - Data export - Disaster recovery

- Restore procedures
- Versioned configuration
- Accidental deletion protection
- Retention policies Display backup status to authorized administrators. Do

not claim that backups are active unless they have actually been configured.

41. REQUIRED TECHNOLOGY STACK — VS CODE + FIREBASE ONLY

Use the following production stack unless a documented technical constraint requires an equivalent actively maintained library. Primary Development Environment - the development agent as the primary AI development and implementation environment - the development agent workspace and file-editing tools for repository changes - the development agent terminal execution for package installation, builds, tests, Firebase CLI commands, and project validation - the development agent browser preview for UI inspection, responsive testing, interaction testing, and smoke testing Node.js Long-Term Support version - npm or pnpm - Git for version control Firebase CLI - Firebase Local Emulator Suite The development agent must work incrementally, keep the repository runnable, avoid unnecessary rewrites, preserve working functionality, and explain important architectural decisions in project documentation. The development agent must not claim successful implementation solely from generated code; it must run the relevant validation
commands and inspect the rendered application whenever the environment permits. Frontend - React - TypeScript - Vite - React Router - Tailwind CSS - shadcn/ui
- Lucide React - Framer Motion only for restrained, purposeful motion
Build the application as a responsive client-side web application and Progressive Web App suitable for Firebase Hosting. Do not use Next.js unless a Firebase-only requirement demonstrably cannot be met with React and Vite and the architectural deviation is documented. The default architecture must not depend on server-side rendering. Firebase Application Platform Use only Firebase services for persistent application infrastructure: - Firebase Authentication - Cloud Firestore - Firebase Storage - Cloud Functions for Firebase, second generation - Firebase Cloud Messaging - Firebase App Check - Firebase Hosting - Firebase Remote Config where useful - Firebase Local Emulator Suite - Firebase Performance Monitoring where useful - Google Analytics for Firebase only when approved Hosting and Deployment Deploy the frontend exclusively with Firebase Hosting. Use: - firebase.json - .firebaserc - Firebase Hosting rewrites for client-side routing - Firebase Hosting preview channels for controlled review deployments Firebase CLI deployment commands - Separate Firebase projects or aliases for development, staging, and production where feasible Do not use any external frontend hosting provider, external serverless-function platform, external edge runtime, duplicate backend, or provider-specific deployment workflow. All trusted backend actions must run through Cloud Functions for Firebase and must verify authentication, authorization, input validity, and App Check where appropriate. Charts - Recharts Forms and Validation - React Hook Form - Zod Server State and Data Access - Firebase modular Web SDK - TanStack Query only where it adds measurable value for asynchronous state, caching, retries, or pagination - Firestore converters for typed documents - Cursor-based Firestore pagination - Controlled real-time listeners Do not place unrestricted Firestore queries directly throughout UI components. Centralize Firebase access in typed service, repository, or feature-layer modules. State Management Use React Context for limited global concerns such as: Authentication - Theme - Language - Current organization configuration Use Zustand only when application complexity justifies a dedicated client-state store.
Do not duplicate Firestore server state unnecessarily in global client state. QR Codes and Barcodes Use actively maintained browser-compatible libraries for: - QR-code generation - Barcode generation - Camera-based QR and barcode scanning - USB barcode scanner input PDF Generation Use: - jsPDF for browser-generated documents when appropriate - Cloud Functions for trusted or resource-intensive report generation when needed Spreadsheet Export - ExcelJS Date Management - date-fns Internationalization Use a production-grade React internationalization solution such as: - i18next - react-i18next All interface text must be stored in translation resources. Do not hard-code Dutch and English labels throughout components. PWA Support Use: - Web App Manifest - Service Worker - Workbox - A

maintained Vite PWA plugin - Firebase Cloud Messaging service worker integration where supported Testing Use: - Vitest - React Testing Library - Playwright - Firebase Emulator Suite - Firebase Security Rules unit testing libraries - Lighthouse for accessibility, performance, and PWA validation Local Browser Verification For each major interface and workflow, the development agent must inspect the running application in a locally launched browser and verify: - Desktop, tablet, and mobile responsiveness - Navigation and protected-route behavior - Form interaction and validation - Loading, empty, error, and permission-denied states Keyboard accessibility and visible focus states - Role-appropriate actions and menu visibility - PWA manifest and service-worker behavior where applicable Code Quality Use: - ESLint - Prettier - TypeScript strict mode - Feature-based folder organization - Environment validation - Reusable typed domain models Centralized error handling - Structured logging in Cloud Functions Select actively maintained libraries compatible with the chosen React, Vite, TypeScript, and Firebase versions. Do not introduce additional hosting providers, backend-as-a-service platforms, duplicate databases, or parallel authentication systems.

42. FIRESTORE DATA ARCHITECTURE

Design scalable Firestore collections such as: - users - roles - permissions - assets - inventoryItems - categories - subcategories - locations - departments employees - assignments - borrowRecords - serviceRequests - repairs - maintenanceSchedules - maintenanceRecords - inventoryMovements - inventoryAudits
- auditItems - suppliers - notifications - activityLogs - reports - settings - attachments - offlineSyncQueue Use references or IDs consistently. Avoid storing
all operational information in one document. Create indexes for common filters and reports. Use Cloud Functions for trusted operations such as: - Sequential reference number generation - Notification generation - Audit sample generation
- Scheduled reminders - Monthly report preparation - Overdue status updates Denormalized statistics -

43. DATA VALIDATION

Implement validation for: - Required fields - Unique asset numbers - Unique serial numbers when applicable - Valid dates - Return dates after borrow dates Warranty dates - Positive quantities - Stock availability - Valid status transitions
- File types - File sizes - User permissions - Location relationships - Category requirements Prevent: - Negative inventory quantities - Duplicate asset numbers
- Assigning disposed assets - Borrowing unavailable assets - Returning an asset
that is not borrowed - Completing audits without required results - -
Assigning one serialized asset to multiple active users Disposing of an asset with an active loan

44. WORKFLOW AUTOMATION

Implement workflow rules such as: - Borrow approval changes a request from pending to approved - Issuing equipment changes the asset status to borrowed
- Returning equipment updates the asset status and return information - Opening a repair changes the asset status to under repair - Scheduling maintenance
generates future reminders - Completing maintenance calculates the next maintenance date - Low stock generates a warning - Warranty expiration generates a notification - Audit discrepancies create corrective action tasks - Disposing an asset closes active assignments and prevents future borrowing - Repair completion prompts the user to select the new asset status - Overdue borrow records are automatically flagged Require confirmation before irreversible actions.

45. FORMS AND USER EXPERIENCE

Forms must include: - Clear sections - Field labels - Required-field indicators Inline validation - Helpful descriptions - Searchable dropdowns - Date pickers File uploads - Image previews - Save as draft - Cancel - Submit - Print - Export when relevant Long forms should use: - Tabs - Steps - Accordions - Progress indicators Prevent accidental data loss by warning users about unsaved changes. Use clear confirmation dialogs before: - Disposal - Archival - Deletion - Stock correction - Audit adjustment - Role changes - s

46. LOADING, EMPTY, AND ERROR STATES

Create polished states for: - Loading - No results - Empty modules - Permission denied

- Offline mode
- Connection problems
- Form errors
- Failed uploads
- Failed exports
- Missing records
- Unexpected server errors
- Synchronization conflicts Error messages must be understandable and

must not expose sensitive technical information.

47. ACCESSIBILITY

Apply accessibility best practices: - Keyboard navigation - Visible focus states - Semantic HTML - Accessible form labels - Screen-reader support - Sufficient color contrast - Accessible dialogs - Accessible tables - Alt text for meaningful images - Reduced-motion support - Touch target sizing - Clear error identification

48. PERFORMANCE

Optimize for: - Fast initial loading - Firestore query efficiency - Pagination Lazy loading - Image optimization - Code splitting - Cached reference data - Efficient dashboard aggregation - Limited real-time listeners - Scalable report generation - Efficient offline caching Do not load the complete inventory database into the browser for normal list pages. Use pagination or cursor-based loading for large collections.

48A. VS CODE IMPLEMENTATION AND VERIFICATION PROTOCOL

The development agent must build the system in controlled, reviewable, and testable increments. For every implementation phase: 1. Inspect the existing repository, package configuration, Firebase configuration, and relevant documentation before modifying files. 2. State the phase objective, affected modules, expected Firebase services, and acceptance criteria. 3. Identify dependencies, security implications, required Firestore indexes, and required emulator services. 4. Create or update only the files required for the phase; avoid unrelated rewrites. 5. Keep all TypeScript code type-safe and compatible with strict mode. 6. Centralize Firebase access through typed services, repositories, converters, or feature modules. 7. Implement the UI, application logic, Firebase integration, validation, authorization, loading states, error states, and audit behavior required for the feature. 8. Run formatting, linting, type checking, relevant unit tests, relevant integration tests, and a production build. 9. Run Firebase Security Rules tests whenever Firestore or Storage access behavior changes. 10. Use the Firebase Emulator Suite to validate authentication, Firestore, Storage, Cloud Functions, and Hosting behavior where relevant. 11. Start the application and inspect the rendered result in a local browser opened from the VS Code development workflow. 12. Test the feature at representative

desktop, tablet, and mobile viewport sizes. 13. Test at least one successful path, one validation failure, one permission failure, and one backend or network failure for each critical workflow. 14. Fix discovered defects before marking the phase complete. 15. Update setup, architecture, schema, security, testing, and deployment documentation. 16. Record unresolved limitations, deferred work, and assumptions explicitly. 17. Provide the exact commands used for local development, testing, emulation, preview deployment, and production deployment. 18. Never expose secrets in source control, logs, screenshots, generated documentation, or frontend code. 19. Never mark a feature complete when only the interface exists without functional Firebase integration and verified business logic. 20. Never claim a test, build, emulator run, preview deployment, or production deployment succeeded unless the corresponding command or verification actually completed successfully. Use Firebase Emulator Suite during development for: - Authentication - Firestore - Storage - Cloud Functions - Hosting where useful Maintain separate environment templates for: - Local development - Staging Production Create a .env.example file containing variable names only. Use Firebase web configuration through environment variables. Treat Firebase web configuration as project identification rather than a privileged server secret, while still preventing accidental cross-environment configuration. Store true secrets, external AI-provider keys, email credentials, and privileged credentials using Firebase-supported secret management for Cloud Functions. Never expose privileged credentials in frontend code. Repository and Change-Control Requirements - Use Git to preserve reviewable changes. - Keep commits or logical change groups focused by implementation phase. - Do not overwrite user-provided files or official templates without preserving the original. - Do not delete working features merely to simplify implementation. - Create migration notes when changing Firestore document structures, indexes, rules, or Cloud Function contracts. - Keep seed data, emulator fixtures, and production data clearly separated. UI Review Requirements Before completing a UI phase, the development agent must verify: The visual result matches the KCS branding and design system. - Tables, forms, cards, dialogs, and navigation are responsive. - Role-restricted controls are absent or disabled appropriately. - Empty, loading, error, offline, and permissiondenied states are present. - Dutch and English translations are available for new
interface text. - Accessibility fundamentals are preserved. - No placeholder data is presented as production data. Before production deployment, the development agent must verify and document: - Correct Firebase project alias - Firestore database location - Authentication providers
- Authorized domains - Firestore indexes - Firestore Security Rules - Firebase
Storage Security Rules - Firebase App Check configuration - Cloud Functions region - Cloud Functions secrets and environment configuration - Hosting rewrites and headers - PWA manifest - Service-worker behavior and cache invalidation
- Firebase Cloud Messaging configuration where used - Environment variables Production build output - Unit, integration, end-to-end, and rules-test results Browser smoke tests - Mobile and desktop responsive checks - Backup status Data migration requirements - Deployment rollback instructions

49. DEVELOPMENT PHASES

Build the application in structured phases. Phase 1 — Foundation - React, TypeScript, Vite, and Firebase project setup Design system - Firebase configuration - Authentication - Roles and permissions
- Main layout - Language system
Phase 2 — Core Inventory - Categories - Locations - Departments - Assets Quantity-based inventory - QR codes - Barcodes - Search - Filters Phase 3 — Operational Workflows - Assignments - Borrow management - Returns - Repairs - Maintenance - Inventory movements Phase 4 — Audits and
Reporting - Random audit generator - Audit verification
- Discrepancy tracking - Reports - PDF export - Excel export - Print layouts
Phase 5 — Intelligence and Automation - Notifications - Scheduled reminders Monthly summaries - AI assistant - Phase 6 — PWA and Mobile Support - Web App Manifest - Service

worker

- Installation experience - Offline interface - Synchronization queue - Camera workflows - Push notifications - Mobile optimization Phase 7 — Production Readiness - Firestore Security Rules - Storage Security Rules - Validation - Testing - Performance optimization - Backup configuration - Firebase Hosting deployment - Firebase environment and project alias configuration - Documentation

50. TESTING REQUIREMENTS

Include: - Unit tests

- Integration tests
- Permission tests
- Firestore Security Rules tests
- Storage Security Rules tests
- Form validation tests
- Inventory movement tests
- Borrow workflow tests
- Return workflow tests
- Audit selection tests
- Audit completion tests
- Report generation tests
- Mobile responsiveness tests
- Accessibility checks
- PWA installation tests
- Offline synchronization tests
- Service worker update tests Test critical scenarios such as:
- Unauthorized users attempting protected actions
- Duplicate asset registration
- Borrowing an unavailable asset
- Negative stock movement
- Overdue return detection
- Random audit generation
- Wrong-location audit results
-
- Losing internet connectivity during an audit
- Synchronization conflicts after reconnecting
- Installing the application on supported devices

51. REQUIRED DELIVERABLES

Produce: 32. Complete application architecture 33. Database schema 34. Firestore collections and document structures 35. Firestore Security Rules 36. Firebase Storage Security Rules 37. User-role permission matrix 38. Page and route structure 39. Reusable UI components 40. Responsive layouts 41. Authentication workflows 42. Asset management workflows 43. Borrow and return workflows 44. Service and repair workflows 45. Maintenance workflows 46. Inventory audit workflows 47. Reporting workflows

48. Notification architecture

49. AI assistant architecture

50. PWA architecture

51. Offline synchronization strategy

52. Firebase Hosting and Firebase CLI deployment configuration

53. Testing strategy

54. Administrator documentation

55. User documentation

56. Setup instructions

57. Environment variable template

58. VS Code implementation instructions

59. Firebase Emulator Suite configuration

60. Firebase project alias strategy for development, staging, and production

52. INPUTS TO BE PROVIDED LATER

The following official materials will be supplied separately: - KCS school logo
- KCS campus photograph - Official KCS service form - Official laptop loan
form when available - School contact details - Final role assignments - Final asset numbering format - Existing inventory data when available - Final report templates when available Create clearly marked placeholders for these materials. Do not invent official school details that have not been supplied.

53. PROGRESSIVE WEB APP, MOBILE INSTALLATION

AND DEVICE SUPPORT The KCS Inventory Management System must be built as a Progressive Web

Application so it can be installed and used as a native-like application on mobile phones, tablets, and desktop computers while remaining fully hosted and deployed through Firebase Hosting. PWA support is a core product requirement and must be included in the frontend architecture, authentication flow, Firebase configuration, security model, testing strategy, deployment workflow, and mobile user experience.

53.1 PWA Requirements

The system must support:

- Installation on Android devices
- Installation on iOS devices through Safari's Add to Home Screen option
- Installation on Windows desktops
- Installation on macOS desktops
- Offline or limited-connectivity operation
- Full-screen standalone application experience
- App-like navigation and transitions
- Responsive portrait and landscape layouts
- Secure authenticated routing in installed mode
- Safe application updates after Firebase Hosting deployments

53.2 Installation Experience

When users open the web application on a supported device, provide: - An Install KCS Inventory App prompt where the browser exposes a native installation event - A custom installation banner where appropriate - Clear manual installation instructions for iOS Safari - An installation help page inside the application - Detection of whether the application is already installed - Detection of whether the application is currently running in standalone mode Do not repeatedly display installation prompts after the user dismisses them. Store the dismissal state for an appropriate period and provide a manual installation
option inside Settings or Help. After installation, the application must: - Open like a native application - Appear on the device home screen or application menu
- Launch without the normal browser address bar where supported - Display a
branded KCS splash screen - Restore the correct authenticated route when possible - Redirect unauthenticated users to the login page - Redirect authenticated users to the correct role-aware dashboard

53.3 Web App Manifest

Create a production-ready web application manifest containing: - App name: KCS Inventory Management System - Short name: KCS Inventory - Description - Start URL: / - Application scope: / - Theme color: #1FAE5B - Background color: #F5FAF7 - Display mode: standalone - Portrait and landscape support Application icons in multiple required sizes - Maskable icons - Screenshots when supported - Appropriate icon purpose values

- Application categories where supported Use the root route as the manifest

start URL. The application must determine the correct destination only after Firebase Authentication has initialized. Provide at least:

- 192x192 icon
- 512x512 icon
- Maskable icon variants
- Apple touch icon
- Favicon assets Do not use a temporary logo in the final production manifest.

53.4 Service Worker and Caching Requirements

Implement a service worker using a maintained Vite-compatible PWA solution. The service worker must support: - Application shell caching - Static asset caching - Faster repeat loading - Offline access to previously viewed approved data - Cache update management after new deployments - Controlled

runtime caching - Offline fallback pages - Safe activation of new application versions Apply controlled caching for: - Application shell assets - Dashboard interface assets - Previously viewed asset lists when permitted - Categories - Locations Departments - Approved reference data - Non-sensitive branding assets Do not manually cache: - Firebase Authentication tokens - Passwords or credentials
- Custom claims - Security-sensitive session information - Private API keys Unauthorized Firestore data - Restricted information unless explicitly approved
for offline use Critical security rules:

- Never bypass Firebase authorization
- Never treat cached data as proof of current authorization
- Refresh security-sensitive data when connectivity returns
- Enforce Firestore Security Rules and Storage Security Rules on all server

synchronization

- Clear protected local data when a user signs out
- Separate cached information between authenticated users where required
- Restrict persistent local caching on shared devices where necessary
- Prevent users from viewing cached data belonging to another account

Firebase Authentication must manage authentication persistence. The service worker must not independently store or manipulate authentication credentials.

53.5 Mobile-First Application Experience

When used as an installed mobile application, optimize for: - Touch-friendly controls - Large scanning buttons - Quick QR-code access - Quick barcode scanning access - Fast asset lookup - Simplified mobile navigation - Optional bottom navigation - Camera-first workflows - Card-based records instead of wide data tables - Mobile-friendly signatures - Mobile-friendly checklists - Clear offline and synchronization indicators - Large touch targets - Safe-area support for modern mobile devices Camera-first workflows must include: - Inventory audits
- Asset verification - Asset registration - Damage reporting Repair evidence - Return-condition evidence Large desktop tables must convert
to structured cards, compact lists, or responsive detail panels on smaller screens.

53.6 Camera and Device Integration

The installed application must support: - Camera access for QR-code scanning
- Camera access for barcode scanning - Image capture for asset registration Image capture for damage capture for audit evidence

- File selection from device storage
- Signature capture where supported
- Image previews before upload
- Image compression where appropriate
- Secure uploads to Firebase Storage Request camera and file permissions

only when they are needed. Display clear permission guidance when access is denied, including instructions for enabling permissions in the browser or operating-system settings. Validate all captured and uploaded files by:

- File type
- File size
- Ownership
- Related record
- User permission
- Firebase Storage Security Rules

53.7 Push Notifications

Use Firebase Cloud Messaging where supported by the browser and operating system. Push notifications may include: - Overdue return alerts - Maintenance reminders - Audit assignments - Audit deadlines - Repair updates - Low-stock

application is closed only where the operating system and browser support background web push. Provide fallback channels when push notifications are unavailable or disabled, including: - In-application notifications - Notification center history - Email notifications when configured Allow users to configure notification preferences according to their role and permissions. Do not claim universal push-notification support across all browsers and operating systems.

53.8 Offline Behavior and Queued Actions

When the device is offline:

- Display a clear Offline Mode indicator
- Allow users to view approved cached information
- Allow users to complete supported queued actions
- Store pending operations securely
- Synchronize pending operations when connectivity returns
- Display synchronization status
- Display failed synchronization records
- Allow authorized users to retry failed operations
- Preserve draft form data
- Warn users when information may be outdated Potential queued actions

include:

- Borrow requests
-
- Audit results
- Status updates
- Location updates
- Damage reports
- Maintenance checklist results Background synchronization must be used

only where supported. The application must also provide:

- Immediate synchronization when connectivity returns
- An in-application retry queue
- Manual retry controls
- Clear failure messages
- Operation timestamps
- Idempotency protection where required Do not allow high-risk actions

such as permanent deletion, approval, role changes, permission changes, disposal approval, or critical stock corrections to complete offline without server validation.

53.9 Synchronization Conflict Management

When offline changes conflict with newer server data: - Do not silently overwrite records - Detect the conflict - Show the conflicting values - Preserve both versions where necessary - Require an authorized user to resolve significant conflicts 30
Record the resolution in the activity log - Record the original client timestamp
- Record the server synchronization timestamp - Identify the user and device
that created the pending change where available Use version fields, last-updated timestamps, transaction identifiers, or equivalent conflict-detection mechanisms.

53.10 Firebase Hosting Deployment Requirements for PWA

Ensure: - Firebase Hosting serves the application over HTTPS - Firebase Hosting rewrites support client-side routing - The service worker is properly registered - The manifest is linked in the application HTML metadata - Icons are optimized and available - Splash-screen assets are configured - Installation requirements are met - Cache versions are controlled - New releases update safely Offline pages are available - Authentication redirects work correctly in installed mode - Firebase Authentication authorized domains are configured - Firebase Cloud Messaging service-worker requirements are configured where used - Firebase App Check is enabled for supported production services - Development, staging, and production Firebase environments use separate configuration where required The application must pass

production-readiness checks for: - Installability - Offline fallback behavior - Accessibility - Performance - Manifest validity
- Service-worker registration - Secure HTTPS delivery - Authenticated routing
- Cache invalidation Use Lighthouse and browser developer tools as validation
aids, but do not rely on a single Lighthouse score as proof that all PWA functionality is correct.

53.11 PWA Testing Requirements

Test: - Android installation - iOS Add to Home Screen instructions - Windows installation - macOS installation - Standalone display mode - Login and logout in installed mode - Session expiration in installed mode - Manifest loading Icon rendering - Service-worker updates - Cache invalidation after deployment
- Offline fallback pages - Cached record access - Sign-out cache clearing - Userto-user cache isolation - Queue creation - Queue retry - Queue failure handling
- Synchronization conflicts

- Camera permission handling
- QR-code scanning
- Barcode scanning
- Firebase Cloud Messaging where supported
- Push-notification fallback behavior

54. OPTIONAL NATIVE APPLICATION PACKAGING

The primary delivery model for the KCS Inventory Management System must remain a Progressive Web Application hosted on Firebase Hosting. The architecture must remain compatible with optional native application packaging when required by St. Kangoeroe Community School. Supported future packaging options may include: - Trusted Web Activity for Android distribution
- Capacitor for Android and iOS packaging - Android APK generation - Android App Bundle generation - Google Play Store distribution - iOS application
packaging through Xcode - Apple App Store distribution - Native access to camera, files, notifications, and device capabilities where required Native packaging must reuse the existing web application and Firebase backend wherever technically appropriate. Do not create a separate duplicate backend for the native application. The web application, installed PWA, Android package, and iOS package must use: - The same Firebase Authentication architecture - The same Cloud Firestore database - The same Firebase Storage configuration - The same role and permission model - The same Cloud Functions - The same audit logging architecture - The same operational business rules - Environment-specific Firebase configuration where appropriate Native packaging must be treated as an optional deployment layer and must not block completion of the primary Firebase-hosted PWA.

54.1 Native Packaging Decision Rules

Use a Trusted Web Activity when: - The Android application mainly presents the hosted PWA

- Deep native integrations are not required
- The PWA already meets installation and performance requirements Use

Capacitor when:

- Deeper native camera integration is required
- Native file-system access is required
- Native notification integration is required
- Additional device APIs are required
- Android and iOS packages must share one web codebase Do not add native

packaging to the first MVP unless institutional distribution requirements justify the additional operational complexity.

54.2 Native Distribution Requirements

Before enabling public app-store distribution, configure and verify: - Application identifiers - Android package name - iOS bundle identifier - Signing certificates
- Android keystore - Apple provisioning profiles - Privacy policy - Terms of use where required - App permissions - Store screenshots - Application icons
- Splash screens - Data safety declarations - App Store privacy declarations
- Production Firebase configuration - Firebase App Check - Push-notification certificates and keys - Deep links - Authentication redirects - Release versioning Crash reporting - Store review requirements Do not claim that an APK, Android App Bundle, or iOS application has been produced until the appropriate native build, testing, signing, and packaging processes have completed successfully.

55. KCS ASSET CODING, QR CODES, FILTERS, AND DISPOSAL

IMPLEMENTATION
SPECIFICATION Continue developing the existing KCS Inventory Management System without removing, overwriting, or breaking its current interface, architecture, Firebase connections, working functionality, or visual identity. This section is mandatory and extends the earlier asset, inventory, QR-code, reporting, security, audit, and workflow requirements. Where this section defines a stricter control, the stricter requirement applies. Implement a production-ready system for:

1. Automatic product and asset coding

2. Recognition and administration of existing KCS code groups

3. Automatic QR-code generation and label printing

4. Advanced searching, filtering, sorting, and saved views

5. Professional lifecycle assessment and physical asset-disposal management

6. Authorization, segregation of duties, and immutable audit logging

7. Number-range control from 01 through 5000

55.1 Official KCS Code Groups

The initial official code groups are:

- KCSMD
- KCSL
- KCSBD
- KCSRT
- KCSPW

Use only active official code groups. Administrators may later add another official group through system

settings. Every code group has an allowed sequence range of 01 through 5000. Examples: KCSMD-01 KCSMD-02 KCSL-125 KCSBD-5000 Accept reasonable legacy input forms such as: KCSMD01 KCSMD-01 kcsmd 01 KCSMD 001

Normalize all valid codes to: PREFIX-SEQUENCE-NUMBER Examples: KCSMD-01 KCSMD-125 KCSMD-5000 Use at least two digits for numbers below 100: 01, 02, 09, 10, 99, 100, and 5000. Store the components separately: codePrefix: KCSMD codeNumber: 147 fullAssetCode: KCSMD-147 Use codeNumber for correct numeric sorting. Never use the visible KCS code as the Firestore document ID. Every asset must have a separate immutable internal identifier.

55.2 Automatic Code-Group Recognition

When a user creates a product, device, material, furniture item, or serialized asset, determine the code group from:

- Category
- Item type
- Subcategory
- Administrator-configured category mapping
- Existing approved coding rules
- Authorized manual confirmation when no mapping exists

Never select a prefix randomly. When a category is mapped to a code group, automatically use that active prefix. Example: Category: Mobile devices Linked code group: KCSMD Generated code: KCSMD-147 When no valid mapping exists:

1. 1. Display a notification.
2. 2. Require selection of an active official code group.
3. 3. Display only groups the user is authorized to use.

4. Allow an authorized user to save the category-to-group mapping for future

records.

Use this message: No default KCS code group has been configured for this category. Select a valid code group to continue.

55.3 Code-Group Administration

Create: Settings → Inventory → Code Groups Each code-group record must include:

- Prefix
- Full name
- Description
- Linked categories and subcategories
- Starting number
- Ending number
- Last issued number
- Used-code count
- Available-code count
- Capacity percentage
- Active or inactive status
- Created date and created by
- Last modified date and modified by

Example: Prefix: KCSMD Starting number: 01 Ending number: 5000 Last issued number: 146 Next available number: 147 Used codes: 146 Available codes: 4854 Status: Active Only administrators or specifically authorized configuration managers may create, edit, activate, deactivate, or remap code groups. Normal users must have read-only access where permitted.

55.4 Transactional Code Generation and Reservation

When a new coded item is created, reserve the first available valid number within the selected code group. Example: Occupied: KCSMD-01, KCSMD-02, KCSMD-03, KCSMD-05 First available: KCSMD-04

Do not merely use the highest number plus one. Reuse gaps only when the number has never previously been issued. An issued code remains permanently unavailable, even if the original asset is archived or disposed of. All code allocation must execute in trusted Cloud Functions for Firebase using Firestore transactions or an equivalently reliable server-side transactional design. The allocation process must:

- Verify authentication, authorization, and App Check where appropriate
- Validate that the prefix exists and is active
- Validate that the requested number range is 01–5000
- Atomically reserve the first never-issued number
- Recheck availability immediately before committing
- Prevent two concurrent users from receiving the same code
- Retry safely after a transaction conflict
- Create the asset and reservation consistently
- Write an immutable audit event
- Be idempotent for retried client requests
- Never trust a client-supplied final KCS code

Create reservation or issuance records so uniqueness is enforced by documentpath design as well as validation. A recommended structure is: assetCodeGroups/{prefix} assetCodeGroups/{prefix}/issuedNumbers/{normalizedNumber} assetCodeReservations/{reservationId} Use a deterministic issued-number document ID such as 0001, 0147, or 5000 to make duplicate creation transactionally impossible. The public code may still display 01, 147, or 5000 according to the formatting rule. After creation, normal users cannot modify an official KCS code. An administrator correction requires:

- A mandatory reason
- A prominent warning
- Additional confirmation
- Verification that the replacement code has never been issued
- Preservation of the previous code in code history
- A complete audit-log entry

Never release the former code for reuse.

55.5 Code-Group Capacity Control

No code group may issue a number greater than 5000. Block examples such as: KCSMD-5001 KCSL-5001 KCSBD-5001 KCSRT-5001 KCSPW-5001 When all 5000 numbers have been issued, block additional creation and display: Code Group Full The maximum capacity of 5000 codes for KCSMD has been reached. No additional code can be generated within this code group. Contact a system administrator. Generate dashboard and administrator warnings at:

- 80% capacity
- 90% capacity
- 95% capacity
- 100% capacity

Examples: KCSMD has used 4000 of 5000 codes.

1000 codes remain available.

Warning: KCSPW has only 250 available codes remaining. Capacity counts must be derived from permanently issued numbers, not only active assets.

55.6 Existing-Code Import and Recognition

Support imports from Excel, CSV, and authorized manual entry. For every imported code:

4. 1. Detect the prefix.
5. 2. Extract the sequence number.
6. 3. Normalize spacing, capitalization, leading zeros, and hyphen placement.

4. Verify that the prefix exists and is active or explicitly approved for legacy migration.

7. 5. Verify that the number is between 01 and 5000.
8. 6. Verify that the full normalized code has never been issued.
9. 7. Block duplicates within the import file and existing database.
10. 8. Place invalid rows in an import-error report.
11. 9. Reserve accepted codes transactionally.
12. 10. Recalculate occupied, available, and next-available numbers.

Normalize:

KCSMD01 KCSMD-01 kcsmd 01 KCSMD 001 To: KCSMD-01 Reject examples including: KCSMD-0 KCSMD-5001 KCSXYZ-01 KCSL-ABC KCSBD--25 Each error row must show:

- Source row number
- Submitted value
- Normalized candidate when possible
- Error type
- Explanation
- Recommended correction

Use a staged import workflow: upload, parse, validate, preview, confirm, transactionally commit, and generate a final result report. Do not partially commit a row without its corresponding code reservation and audit record.

55.7 Permanent Code Retention and No Reuse

Once issued, a code may never be assigned to another item, including after the original item is:

- End of useful life
- Sold
- Donated
- Traded in
- Recycled
- Destroyed
- Lost
- Stolen
- Archived
- Soft deleted after an exceptional administrative correction

Retain the permanent historical relationship between the code and original asset. Relevant lifecycle statuses include:

Active In Use In Stock Under Repair Disposal Requested End of useful life Sold Donated Traded In Recycled Destroyed Lost Stolen Archived Never hard-delete a coded asset through a normal application workflow.

55.8 Automatic QR-Code Generation

Immediately after a coded asset is successfully created, generate a unique QR code linked to:

- Immutable internal asset ID
- Official KCS code
- Secure asset-detail URL
- Current authorization-controlled detail experience

Example route: https://[KCS-DOMAIN]/inventory/assets/[UNIQUE-ASSET-ID] The QR payload must not expose confidential personal, authentication, or security information. Prefer a secure HTTPS detail URL containing an opaque internal identifier. The QR code must:

- Be generated automatically
- Be available as PNG
- Be available as SVG where supported
- Be printable
- Be suitable for asset labels
- Appear on the asset detail page and print view
- Be regeneratable without changing the asset identity
- Continue opening the same asset after location, department, assignment,

description, condition, or status changes Store QR metadata separately where useful, including version, payload type, generated date, generated by, checksum, and active status. Regeneration must create an

audit event.

55.9 QR Asset Labels and Batch Printing

Create professional printable labels containing at least: KCS logo KCS Inventory KCS code Item name QR code Serial number Support:

- Small label
- Standard asset label
- A4 label sheet
- One label per page
- Multiple labels per page
- Configurable margins and label dimensions
- Print preview
- Batch selection and printing

Example: KCS INVENTORY KCSMD-147 Dell Latitude 5420 [QR CODE] Serial number: DL5420-82931 Use the official configurable KCS branding. Ensure printed QR codes remain scannable at their intended physical size.

55.10 QR Scanning and Secure Detail Access

Support scanning with:

- Mobile phone cameras
- Tablet cameras
- Laptop cameras
- External USB or Bluetooth QR/barcode scanners that emulate keyboard

input After a successful scan, open the corresponding asset detail page. According to permissions, display:

- KCS code
- Item name
- Category and subcategory
- Serial number
- Location and department
- Assigned user
- Status and condition
- Purchase date
- Latest inspection
- Maintenance status
- Lifecycle Assessment status

Never expose restricted personal data to unauthorized users. Authentication and current permissions must be evaluated when the URL is opened; possession of a QR code is not authorization.

55.11 Professional Inventory List

The primary asset list must support configurable columns including:

- Selection checkbox
- QR-code indicator
- KCS code
- Item name
- Category
- Subcategory
- Item type
- Brand
- Model
- Serial number
- Location
- Department
- Assigned user
- Stock status
- Asset status
- Condition
- Purchase date

- Last updated
- Actions

Support:

- Horizontal scrolling
- Sticky headers
- Appropriate sticky identifying columns on large screens
- Cursor-based pagination
- Configurable results per page
- Quick actions
- Export
- Bulk selection
- Mobile card or compact-list layouts
- User-specific visible-column preferences

Do not load the complete asset database into the browser.

55.12 Global Search

Provide one global inventory search supporting partial and normalized matching across:

- KCS code
- Item name
- Serial number
- Brand
- Model
- Category
- Subcategory
- Department
- Location
- Assigned user
- Supplier
- Notes where permitted

Examples: Search: KCSMD Result: authorized items using the KCSMD prefix Search: 147 Result: relevant codes and serial numbers containing 147 Implement a Firestore-compatible search architecture. Do not imply that unrestricted full-text search is native to Firestore. Use normalized searchable fields, prefix tokens, exact-match indexes, scoped queries, or a documented Firebase-compatible search strategy that does not introduce another operational database.

55.13 Advanced Filters and Numerically Correct Sorting

Provide an advanced filter panel with removable active-filter chips. Alphabetical controls:

- Item name A–Z
- Item name Z–A
- Begins with selected letter
- A–Z letter bar

Code-group filters:

- KCSMD
- KCSL
- KCSBD
- KCSRT
- KCSPW
- Any future active official group
- All code groups

Code sorting:

- Lowest number first
- Highest number first
- Prefix A–Z
- Prefix Z–A
- Full code ascending
- Full code descending

Numeric order must be correct: KCSMD-01 KCSMD-02 KCSMD-10 KCSMD-100 Never perform lexical sorting that places KCSMD-02 after KCSMD-100. Item-type filters:

- Device
- Product
- Material
- Furniture
- Tool
- Accessory
- Component
- Consumable
- Other

Stock-status filters:

- In stock
- Partially issued
- Fully issued
- Out of stock
- Reserved
- Ordered
- Received
- Returned
- Defective
- Under repair

Asset-status filters:

- Active
- In use
- In storage
- Under repair
- Out of service
- Disposal requested
- Awaiting inspection
- Awaiting approval
- Approved for disposal
- End of useful life
- Sold
- Donated
- Traded in
- Recycled
- Destroyed
- Lost
- Stolen
- Archived

Condition filters:

- New
- Excellent
- Good
- Fair
- Damaged
- Defective
- Unsafe
- Economically irreparable

Additional filters:

- Category and subcategory
- Department and location
- Assigned user
- Supplier
- Brand and model
- Purchase year
- Warranty status

- Maintenance required
- Low stock
- Missing QR code
- Missing serial number
- End of useful life
- Not end of useful life
- End of useful life but still in use
- Date added
- Last updated

Allow multiple simultaneous filters. Provide: Apply Filters Clear Filters Save Filter View Design Firestore indexes and query constraints for supported combinations. When a combination cannot be served efficiently, present a clear limitation or use a trusted report-generation workflow rather than downloading all records.

55.14 Saved Views

Allow users to save private filter, sort, and column configurations, including examples such as: All laptops Low stock Defective devices KCSMD ascending Items without QR labels End of useful life but still in use Awaiting disposal Administrators may publish shared organization-wide views. Saved views must preserve only filters and display configuration the user is authorized to access. Revalidate permissions whenever a view is opened.

55.15 Bulk Actions

Authorized bulk actions include:

- Generate QR labels
- Print QR labels
- Export to Excel
- Export to CSV
- Change location
- Change department
- Change status
- Schedule inspection
- Request disposal
- Archive

Enforce permissions server-side. High-risk bulk actions require:

- Clear warning
- Confirmation
- Mandatory reason
- Affected-record summary
- Audit-log registration for each asset or a traceable batch event linked to

per-record changes

- Transactional or resumable processing with visible partial-failure reporting

55.36 VS Code Implementation Protocol for This Section

The development agent must implement this section in controlled increments:

1. Inspect the current repository, Firestore model, Security Rules, Cloud

Functions, routes, components, and existing inventory records.

2. Document the current asset, category, stock, status, and audit

fields before modifying the schema.

3. Confirm an actual backup, export, emulator fixture, or documented restore

point before production data migration. Do not claim a backup exists unless verified.

13. 4. Design migration and rollback procedures for existing items and codes.
14. 5. Preserve all working functionality and visual identity.

6. Add typed domain models, converters, validation schemas, repositories,

and reusable components.

7. Implement code allocation only in trusted Cloud Functions using transactions

and idempotency protection.

8. Add deterministic issued-number records and migration logic for existing

KCS codes.

9. Implement Security Rules and Storage Rules before exposing protected

workflows.

10. Test range boundaries, missing-number allocation, imports, concurrent

creation, permissions, and disposal gates with Firebase Emulator Suite. 11. Run linting, formatting checks, strict TypeScript checking, unit tests, integration tests, Rules tests, end-to-end tests, and a production build.

12. Inspect the rendered application in desktop, tablet, and mobile browser

sizes. 13. Update architecture, schema, setup, administrator, user, migration, testing, deployment, and rollback documentation.

14. Provide exact Firebase CLI commands for emulation, index deployment,

Rules deployment, Functions deployment, Hosting preview, and production deployment.

15. Do not mark the section complete until Firebase integration, authorization,

audit logging, error handling, and actual verification are complete. At completion, report:

- Modified files
- New files
- Database and schema changes
- New collections, subcollections, indexes, and migrations
- Cloud Functions added or changed
- Firestore and Storage Rules changes
- Tests executed and their actual results
- Browser and responsive checks performed
- Known limitations
- Required administrator configuration
- Deployment and rollback steps

56. FINAL PRODUCT STANDARD

The completed KCS Inventory Management System must resemble a professional Enterprise Asset Management and ICT Inventory platform used by a large organization while remaining simple enough for daily use by school ICT and warehouse employees. The final product must combine: - KCS branding - Premium enterprise UI
- Controlled glassmorphism - Dutch and English support - Secure role-based
access - ICT asset tracking - Warehouse inventory control - Equipment assignments - Borrow and return management - Digital service forms - Repair tracking - Preventive maintenance - QR-code and barcode scanning - Random physical inventory audits - Complete audit trails - Intelligent notifications - Operational reporting - Monthly inventory summaries - AI-assisted
inventory search and analysis - Scalable Firebase architecture - Firebase Hosting deployment - Progressive Web App installation Mobile camera workflows - Controlled offline capability - Secure synchronization - Optional future native packaging The application must function as: - A complete Firebase-hosted web application - An Android-installable PWA - An iOS home-screen application - A desktop-installable application - A hybrid enterprise system with controlled offline capabilities - An architecture that can later support optional Android and iOS store packaging The installed experience must feel like a professional mobile application, not merely a website saved to a home screen. Prioritize: - Data accuracy - Accountability - Security - Usability - Maintainability - Performance - Accessibility - Long-term scalability Do not sacrifice operational clarity for visual decoration. Build the platform as a production-ready institutional system, not as a conceptual prototype.